

What is a graph? [UDL-13]

Graphs coded into matrices

Graphs are powerful tools in modeling interactions/relationships between different elements. By modeling such dependencies/interactions in a reliable way, one can derive rules, predictions or new knowledge about the system in hand.

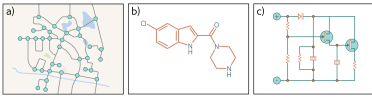


Figure 1. Real-world graphs. Some objects such as a) road networks, b) molecules, and c) electrical circuits are naturally structured as graphs.

- Social networks are graphs where nodes are people and the edges represent friendships between them.
- The scientific literature can be viewed as a graph in which the nodes are papers and the edges represent citations.
- Wikipedia can be thought of as a graph in which the nodes are articles and the edges represent hyperlinks between articles.
- Computer programs can be represented as graphs in which the nodes are syntax tokens (variables at different points in the program flow) and the edges represent computations involving these variables.
- Geometric point clouds can be represented as graphs. Here, each point is a node and has edges that connect to other nearby points.
- Protein interactions in a cell can be expressed as graphs, where the nodes are the proteins, and there is an edge between two proteins if they interact.

More examples

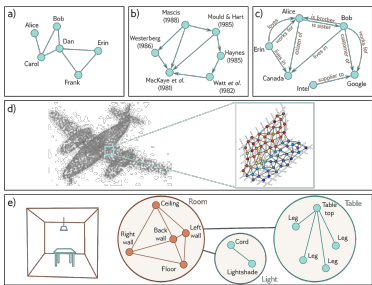


Figure 2. Types of graph. a) A social network is an undirected graph; the connection between people are symmetric. b) A citation network is a directed graph; one publication cites another, so the relationship is asymmetric. c) A knowledge graph is a directed heterogeneous multigraph. The nodes are heterogeneous in that they represent different object types (people, places, companies) and there may be multiple edges representing different relations between each node. d) A point set can be converted to a graph by forming edges between nearby points. Each node has an associated position in 3D space and this is termed a geometric graph (adapted from Ili et al. 2021). e) The scene on the left can be represented by a hierarchical graph. The topology of the room, table, and light are all represented by graphs and these graphs form nodes in a larger graph representing object adjacency (adapted from Fernández-Madruga & González 2002).

Graph representation into a matrix

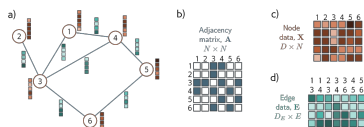


Figure 3. Graph representation. a) Example graph with six nodes and seven edges. Each node has an associated embedding of length five (brown vectors). Each edge has an associated embedding of length four (blue vectors). This graph can be represented by three matrices.

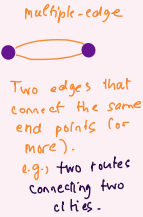
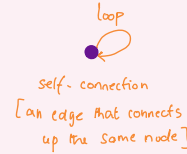
Graph types

1. Directed vs undirected/ binary vs weighted



2. Simple graph

A graph is simple if it has no loops or multiple edges.



3. Complete graph

A complete graph has each of its nodes connected to all other nodes.

→ every two nodes are adjacent.



8. Hypergraphs

→ models high-order relationships among nodes.

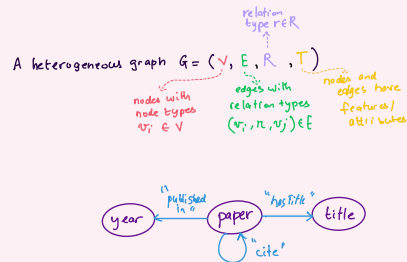
A hypergraph H is a generalization of a graph in which an edge can join any number of vertices.

→ It is a set of non-empty subsets (or hyperedges).

	v_1	v_2	v_3	v_4
v_1				
v_2				
v_3				
v_4				

incidence matrix

7. Heterogeneous graphs

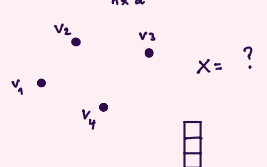


6. Annotated/decorated graphs

A graph G is annotated or decorated if its nodes can hold attributes / features.

$$G = ((V, X), E)$$

size n site $n \times d$ max size?



4. Subgraphs

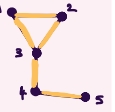
examine the structure of graphs that lie within a given graph.

A graph H is called a **subgraph** of G if every vertex and edge of H belong to G .

A graph G is a **subgraph** of itself.

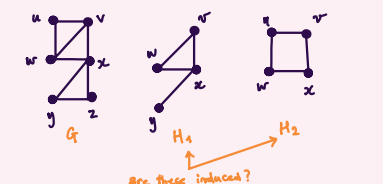
→ all other subgraphs are proper subgraphs.

If H is a subgraph of G having the same vertices as G , then H is a **spanning subgraph** of G .



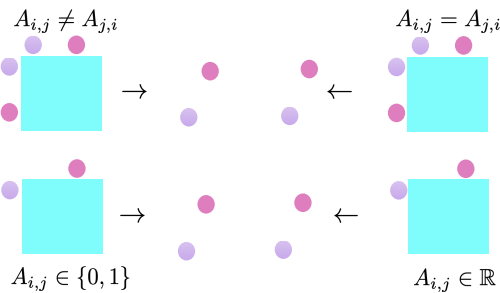
5. Induced subgraphs

Given a non-empty set $S \neq \emptyset$ of nodes of G , the subgraph $G[S]$ of G induced by S has vertex set S where two vertices u and v are adjacent if and only if (u, v) are adjacent in G .



Graph (matrix) properties

1. Connectivity: symmetric vs asymmetric matrix (weighted or binary)

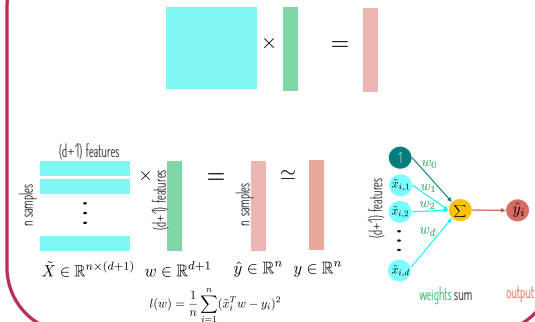


2. Scale/size (density, sparsity) & permutation invariance

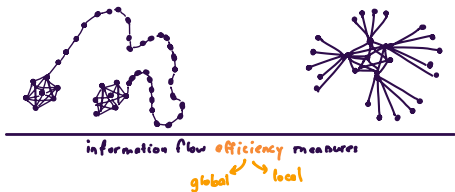
$$A \in \mathbb{R}^{N \times N} \quad A_{i,j} \neq 0$$

$$X' = XP \quad A' = P^T A P$$

3. The power of integrating information



8. Efficiency and resilience $\rightarrow$ GraphT_5 GraphT_6

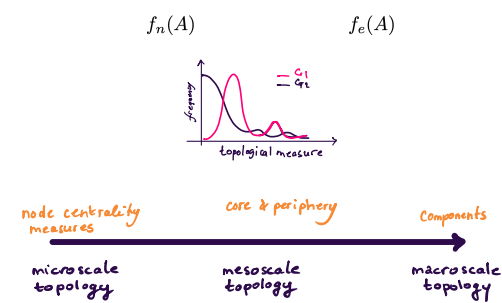


- * How the topological properties of graphs constrain the way in which information flows through the graph?
- * Which specific graph properties can be used to gain insight into information processing by graph units (nodes/edges)?

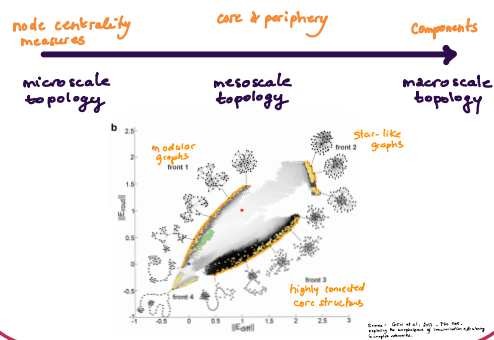
Adjacency matrix, A
 $N \times N$

1	2	3	4	5	6
1					
2					
3					
4					
5					
6					

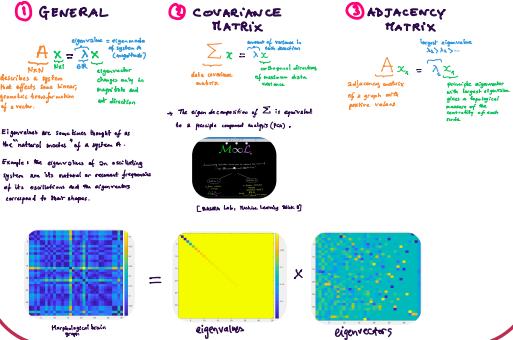
4. Topology $\rightarrow$ watch GraphT_2



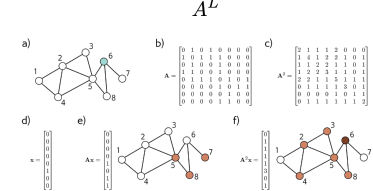
7. Organization pattern



6. Spectra of the matrix (eigen decomposition)

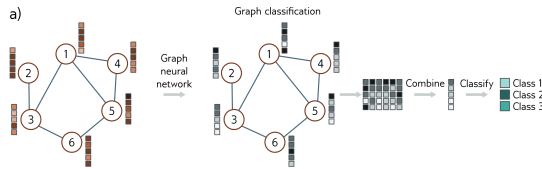


5. Walks (powers of matrices)

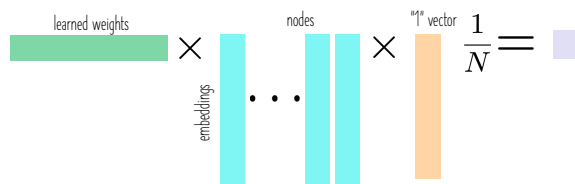


Learning tasks

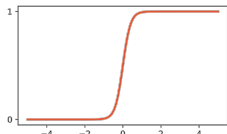
Graph classification



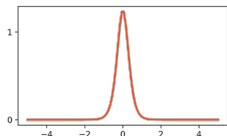
$$y = \text{sig} [\beta_K + \omega_K \mathbf{H}_K \mathbf{1}/N]$$



Logistic activation (sigmoid)

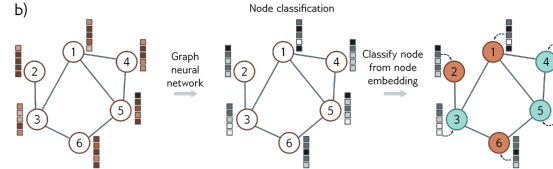


$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

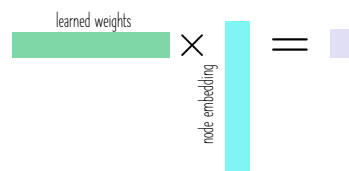


$$\frac{d}{dx} \sigma(x) = \sigma(x) \cdot (1 - \sigma(x))$$

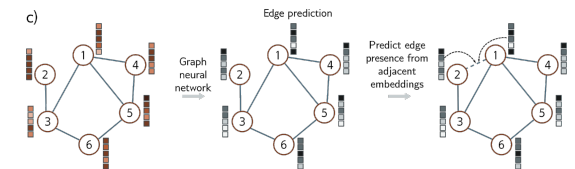
Node classification



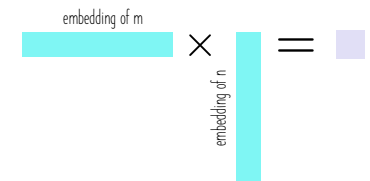
$$y^{(n)} = \text{sig} [\beta_K + \omega_K \mathbf{h}_K^{(n)}]$$



Edge prediction



$$y^{(mn)} = \text{sig} [\mathbf{h}^{(m)T} \mathbf{h}^{(n)}]$$



Generalized $\left\{ \begin{array}{l} \text{node-level} \\ \text{edge-level} \\ \text{graph-level} \end{array} \right\}$ prediction

